VSP Pipeline — User Guide

Audience: Operators of the VSP desktop UI on Windows 11 + RTX 5090 laptops. **System:** Argos VSP — visual speech (lip-reading) pipeline with LLM context. **Version:** May 2026.

---

1. What VSP does

VSP is a lip-reading system: it watches a person speak in a video and produces a written transcript using **only the visual signal from the mouth** (audio is not used for transcription — Whisper audio is used once, separately, as a teacher signal during segment review). A visual encoder turns lip motion into features; a Large Language Model (currently Llama-2-7B with a project-specific adapter) reads those features and emits English text. The LLM contributes language context — grammar, plausible word sequences, common phrases — which is how the system recovers words that are visually ambiguous (many English visemes look identical on the lips). The result is a per-segment transcript plus quality metrics that tell you how much to trust each segment and each word inside it. Everything runs locally on the laptop's GPU.

Baseline performance on 1,497 wild YouTube segments (the standard evaluation set):

- Mean WER **64.1%** (top-1 displayed output) / **63.8%** (MBR aggregated output, the production default).
- Mean Intelligibility Score **2.532 / 5.0** (top-1) / **2.547 / 5.0** (MBR).
- Useful content rate, as judged blindly by an LLM (NIV Y+P): **68.4%** (top-1) / **71.1%** (MBR; +2.7 pp, paired McNemar p=0.0002).
- Deterministic IS-NIV cut at IS ≥ 2.00: **61.7%** (top-1) / **61.9%** (MBR) — used as a cheap proxy when re-judging is too costly.

These numbers are the system's wild-video baseline. Your individual videos may do better or worse depending on the speaker, framing, and domain vocabulary.

---

2. First-time setup recap

The image is installed; this section is the operator refresher.

1. **Desktop shortcut.** Double-click the **VSP Pipeline** icon on the Public Desktop (`C:\Users\Public\Desktop\VSP Pipeline.lnk`). The shortcut runs a `.bat` wrapper that starts a PowerShell window.
2. **PowerShell window opens.** The window prints container startup progress. Leave it open — closing it stops the server. The window stays attached to `docker logs -f` so you can monitor the run.
3. **Browser auto-opens** at `http://127.0.0.1:8080`. Use `127.0.0.1`, **not** `localhost` — Windows resolves `localhost` to IPv6 `::1` and Docker Desktop's IPv6 forwarding is unreliable.
4. **One-time CUDA JIT compile (10–15 minutes).** The first decode on this laptop will pause for ~10–15 minutes with no log output between the words "starting decode" and the first segment result. This is **not a freeze.** PyTorch is compiling PTX kernels to native sm_120 SASS for the RTX 5090 (Blackwell). The compiled kernels are cached to `%USERPROFILE%\.nv` so every subsequent run reuses them and is fast. If you delete the cache directory or move to a new laptop, the 10–15 minute pause comes back once.

> ***What you'll see:*** *the PowerShell window goes quiet; the browser shows the "Processing" screen with progress at 95–98% and no visible movement. Both are correct. Wait it out.*

---

3. Putting videos in

There are three ways to get videos into the pipeline; pick whichever fits your workflow.

3a. File Explorer

Open File Explorer and navigate to:

```
%USERPROFILE%\vsp-input
```

(That expands to e.g. `C:\Users\you\vsp-input`.) Drop video files directly into the folder. The UI rescans this folder when you click **Inspect Videos**.

3b. Drag-and-drop into the browser

Drag video files from File Explorer directly onto the VSP browser window. They're uploaded into the same `%USERPROFILE%\vsp-input` folder and appear in the count immediately.

3c. Copy / paste a path

The input-folder display on the welcome screen shows the canonical path. Copy any video into that path from PowerShell, robocopy, or a mapped network share. The UI picks it up on the next **Inspect Videos**.

Archiving — `.excluded` sibling folder

There is a sibling folder `%USERPROFILE%\vsp-input.excluded` (note the dot). Move files there to keep them out of the pipeline without deleting them. The UI ignores anything inside `.excluded`. This is the recommended way to keep an "archive of processed videos" near the input folder.

Supported inputs

- Container formats — eleven extensions accepted:
- **Modern web / mobile**: `.mp4`, `.mov`, `.mkv`, `.webm`, `.m4v`.
- **Camcorders (AVCHD / MPEG-TS)**: `.mts`, `.m2ts`, `.ts` — Sony / Panasonic / many handheld camcorders.
- **Legacy**: `.avi`, `.wmv` (Windows Media), `.flv` (Flash).
- 10-bit / HDR videos are auto-converted to 8-bit BT.709 on the way in (GPU-accelerated where possible). Camcorder `.MTS` / `.m2ts` files are re-muxed to `.mp4` by the normalization stage; original files stay untouched in your input folder.
- Anything ffmpeg can decode that's not in this list — drop it in and the normalization stage will tell you in the log if it can't read it; nothing else in the input folder is affected.
- USB drives: copy the videos onto `C:` first. Docker Desktop's bind-mount of removable drives is unreliable.

4. The two-stage flow

VSP runs in two distinct stages, with a manual review step between them. This is deliberate: the LLM decode is the expensive operation, so you fix transcriptions *before* paying for it.

Stage 1 — Segmentation + Whisper auto-transcribe (~1–2 min)

When you click **Start Processing** on the welcome screen:

1. Each video is cut into segments (3 to 12 seconds by default; the 12 s ceiling matches the LLM's context window).
2. Audio is extracted and Whisper transcribes each segment.
3. A **Segment Review** screen appears, showing every segment, its Whisper transcription, and a play button.

Stage 1 is fast (under 2 minutes for a typical 10-minute source video) and is the same on every laptop.

Stage 2 — LLM decode (~3 min after JIT)

When you click **Continue Pipeline** on the Segment Review screen:

1. The video frames around the mouth are cropped and aligned.
2. The visual encoder extracts features; k-means quantises them.
3. The LLM (Llama-2-7B + project adapter) reads the feature sequence and emits text.
4. Per-word confidence, beam agreement, and the n-best aggregation are computed.
5. `report.html`, `report.csv`, the burned video, and the segment sidecars are written under `outputs\<run-id>\`.

On a warm RTX 5090, Stage 2 takes ~3 minutes for a typical run of ~50 segments. On the **first ever decode** (cold JIT cache) add 10–15 minutes; see §2.

Why two stages?

- Stage 1 produces a Whisper "teacher" transcript per segment. This is the reference text used to compute WER on your specific videos. If Whisper is wrong, every quality metric for that segment will be wrong too — so you get a chance to fix it before paying for decode.
- Stage 1 is also where you choose the k-means model (§6). That choice changes Stage 2's accuracy, and it needs to be made after you see how many segments came out of segmentation.

5. Segment Review — when to edit Whisper transcriptions

Whisper is the audio ASR system that produces a per-segment "reference" transcript. It is **only used as the comparison text** for WER / NIV / IS metrics — the visual model never sees it. But because every quality number you read in the final report is computed against the Whisper text, **a wrong Whisper transcript gives you a wrong report**.

Edit a segment's Whisper text when:

- **Proper nouns are obviously wrong.** Whisper often invents phonetically-plausible nonsense for names it has never seen ("Lee Sin Geng" instead of "Li Shengang"). The visual model may actually have got the name right; you need to fix the reference so the report shows a green tick, not a red WER.
- **Numbers and dates.** Whisper hallucinates digits routinely (writes "21" for "twenty-one", or "2003" for "twenty-oh-three"). Numbers are also the highest-cost class for the visual model — see §8 — so a clean reference here matters more than anywhere else.
- **Low-confidence regions.** Whisper shows lower confidence on unclear audio. If the auto-transcription reads as gibberish to you, it almost certainly is.

How to spot bad Whisper output

- **Repetition loops** ("the the the the"). Whisper falls into these on long silences.
- **Sudden language switch.** Whisper occasionally jumps into German or Welsh on a few seconds of accented English.
- **Hallucinated boilerplate.** "Subscribe to my channel" or "Thanks for watching" appearing on segments that don't contain those words — Whisper memorised them from YouTube training data.
- **Length mismatch.** A 10-second segment that comes back with one word, or a 3-second segment that comes back with a paragraph, is almost always wrong.

You don't need to fix every segment — Whisper is right ~85–90 % of the time on clear English audio. Focus on the failures and the segments you care most about quoting from.

Transcriptions persist

Your edits are saved to `%USERPROFILE%\vsp-input\.transcriptions\` as `.wrd` files (one per segment). The next time the same segment file runs through the pipeline, the saved transcription is reused automatically and Whisper is skipped for it. This is how you build a clean reference set for repeat-processed videos.

Inject from audio (when you have a separate recording)

If the video has bad or missing audio but you have a **clean separate audio recording** of the same speech (e.g. a microphone the camera didn't capture, or a podcast version of a talking-head video), use the **"Inject from audio…"** button on the Segment Review screen.

The modal asks for:

- **Video** — pick the parent video from the dropdown.
- **Audio file** — drag in `.wav`, `.mp3`, `.m4a`, or `.flac`.
- **Audio start (`T_a`)** and **Video start (`T_v`)** — two offsets that declare "audio time `T_a` corresponds to video time `T_v`". Leave both at `0` if the audio and video are perfectly aligned at their starts.
- **Whisper model** — five options in the dropdown: `tiny` / `base` / `small` / `medium` (default) / `large`. Larger is slower but better on accents and noise; `medium` is the sweet spot for clear English speech.

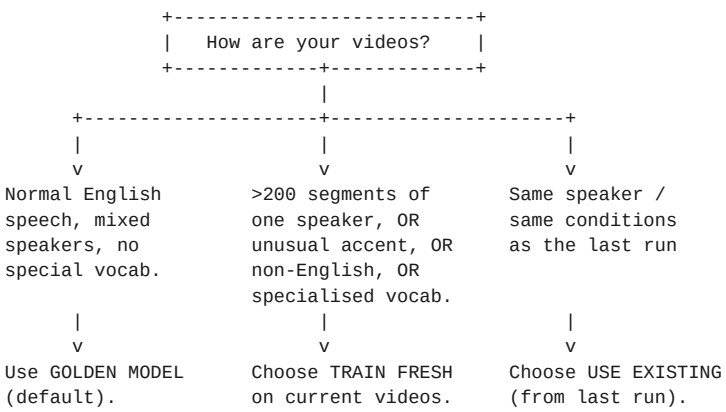
The tool clips the audio per segment, runs Whisper on each clip, and writes the result into `.transcriptions/` with a new badge type **[AUDIO-INJECTED]** (distinct from `[AUTO]` Whisper-on-video and `[MANUAL]` typed-by-hand). Re-runs of the pipeline reuse these the same way they reuse manual transcriptions.

Same flow available from the CLI for batch / scripted use — see audio-injection.md.

---

6. K-means model choice

The k-means model is a small clustering step that quantises the visual encoder's features before they go to the LLM. You pick the mode on the Segment Review screen, in the **K-means Model Options** panel.



Mode A — Use Golden Model *(default)*

Pick this for **normal English speech**: news, interviews, lectures, talking-head YouTube videos, mixed speakers. The "golden" k-means was trained on a balanced English-speech corpus and generalises well. This is the default option on the radio button and what you should use unless one of the other two cases applies.

Mode B — Train Fresh on current videos

Pick this when you have **more than ~200 segments** of one of the following and want the k-means to specialise to it:

- A single speaker, recorded across one or more videos (the model learns the speaker's mouth-shape distribution).
- An unusual accent that English-corpus k-means doesn't represent well (Scottish, Indian English, heavy regional accents).
- A non-English language (note: the LLM is English-only, so the decode will still produce English — but the k-means features will be better matched to the source language's visemes).
- Specialised vocabulary where the same words recur (medical, legal, sports commentary, gaming).

Below ~200 segments the fresh k-means under-fits and you'll get worse results than the golden model. The UI shows a warning if you pick **Train Fresh** with fewer than 200 segments.

Mode C — Use Existing (from last run)

Pick this when you're **re-running the same speaker / same conditions** as the previous pipeline run and want to save the ~5 minutes of k-means training time. The k-means files from the last run are reused as-is. This is purely a speed optimisation; the quality is whatever the last run's k-means was.

Quick decision

- Don't know? → **Golden Model.**
- Got a big single-speaker dataset (≥200 segs)? → **Train Fresh.**
- Re-running the same material? → **Use Existing.**

---

7. Reading the report

After Stage 2 completes, open `%USERPROFILE%\vsp-output\<run-id>\client_outputs\report\report.html` in any browser (on a fresh install `<run-id>` is the timestamp directory created at run time). The columns are described below in left-to-right order. Numbers and reliability values cited here are from the canonical project benchmark (1,497 wild-YouTube segments, May 2026).

Top of the report — overall summary

A header line shows the aggregate numbers across all segments in the run: WER, WWER, NEA F1, IS, Mode, Segment count. Use these to gauge the run as a whole.

Per-segment columns

Column	What it means	How to read it
ID	Segment identifier (`videoname_startms_endms`)	Lets you locate the source clip.
Reference	The Whisper transcript you reviewed in §5	This is the ground truth for the WER/WWER/IS columns.
Hypothesis (Accuracy)	What the model said, coloured by per-word accuracy vs the reference	**Green = matches reference, red = mismatch. Visual cue for quick scanning of failures.**
Hypothesis (Confidence)	The same text, coloured by per-word **confidence**	This is the operationally important column. See colour key below.
WER	Word Error Rate on this segment (lower = better)	The classic ASR metric. Baseline mean is 64.1 %. <30 % = excellent, 30–60 % = useful, >100 % = hallucinated.
WWER	Weighted WER — high-value tokens (names, numbers, content words) cost 2×.	Captures "wrongness that matters" better than plain WER. Baseline mean 60.5 %.

Column	What it means	How to read it
NEA Recall	Named-Entity Accuracy: % of names / dates / numbers in the reference that the hypothesis recovered.	Baseline NEA F1 is 38.9 % — entities are missed in ~85 % of segments. Treat low NEA as a red flag for the segment's facts.
IS	Intelligibility Score 1.0–5.0	The composite quality score. See tier table below.
Sent Conf	Sentence Confidence — the segment's mean per-word probability (0–1)	Drives the per-word colour rendering (§7.1).
Tier	**Trust / Salvage / Strip — the segment-level reliability category**	The first thing you should look at. See §7.2.

7.1 The coloured words — per-word confidence bands

Each word in the **Hypothesis (Confidence)** column is painted one of three colours under the joint *confidence + beam-agreement* rule (production default since May 2026):

Colour	Rule	What it promises	Empirical P(correct)
Green — TRUST	`top1_conf ≥ 0.95` AND `beam_agreement ≥ 0.80`	"Trust without review."	~94% inside Trust segments; ~80% inside Salvage; ~37% inside Strip (= why Strip strips the colours).
Yellow — SALVAGE	`top1_conf ≥ 0.65` AND `beam_agreement ≥ 0.50`	"Could be right, worth a glance."	~65% inside Trust; ~41% inside Salvage; ~17% inside Strip.
Red / purple — STRIP / ignore	Everything else	"Model is guessing — likely wrong."	~39% inside Trust; ~20% inside Salvage; ~7% inside Strip.

Numbers and proper nouns are capped at yellow regardless of the model's reported confidence — see §8 for why.

The "P(correct|green) ≈ 92 %" headline number you may have seen in sales material refers to the overall green-word reliability under the old confidence-only rule. Under the current joint rule that climbs to **89.8 % overall**, and to **94 % inside Trust segments** / **80 % inside Salvage segments**. The numbers are stratified for a reason: a green word inside a Strip segment is roughly half as reliable as the same green word inside a Trust segment. **Always read the tier first.**

7.2 NIV tier (Y / P / N)

NIV stands for *Net Intelligibility Verdict*. It's a 3-level collapse of the LLM-judge gold standard:

NIV	Label	Definition	IS threshold
Y	*Yes* — meaning clearly conveyed	The hypothesis preserves the key facts of the reference.	IS ≥ 3.80

NIV	Label	Definition	IS threshold
P	*Partial* — some meaning preserved	Structure or topic recoverable; specific words or facts lost.	2.00 ≤ IS < 3.80
N	*No* — meaning lost	Wrong topic, empty output, or hallucination.	IS < 2.00

The thresholds are calibrated against the Claude Opus 4.6 blind human-style judge on the full 1,497-segment set ($\kappa = 0.71$ for Y, $\kappa = 0.82$ for Y+P).

Baseline rates: **24 % Y**, **62 % Y+P** (any useful output), **38 % N**. Your run's rates will be visible in the per-segment IS distribution.

7.3 Intelligibility Score (1–5)

IS is the composite score (semantic similarity, phonetic similarity, WER, WWER, NEA, length ratio) used for the NIV tier. The tier table:

IS range	Tier	Label
4.00 – 5.00	5	Excellent
3.00 – 3.99	4	Good
2.00 – 2.99	3	Fair
1.00 – 1.99	2	Poor
0.00 – 0.99	1	Failed

The "captured" rate in the run summary counts segments with IS ≥ 3.0 (legacy threshold). The newer NIV-Y rate (IS ≥ 3.80) is the stricter "clearly conveyed" bar.

---

8. Common failure modes

The visual signal is fundamentally ambiguous (English has ~14 distinct visemes for 44 phonemes). The system handles most of the ambiguity via language context, but three failure classes survive and deserve operator awareness.

8a. Hallucinated output (fluent but wrong)

The model generates grammatical English that has no basis in what the person actually said. On the wild-video baseline, **20.5 % of segments** are hallucinations (WER ≥ 100 %). These are the most dangerous failure mode because the output *looks* trustworthy at a glance. The defence:

- Check the **Sent Conf** column. Hallucinations typically score below 0.50.
- The Tier column flags these as **Strip**.
- Per-word colouring is suppressed inside Strip segments. If you see a segment with no green/yellow/red colouring and italic grey text, it's a Strip segment — don't quote individual words from it.

8b. Numeric confusion (billion ↔ million)

Numbers are visually almost identical (lips form the same shapes for "M" and "B"). The model frequently substitutes one for the other and reports **high confidence** on the substitution. Observed examples from the calibration set:

- "**billion** → million" (model reported 0.965 confidence — off by 1000×).
- "**1024** → 24" (0.958 confidence).
- "**2011** → 2000" (0.894 confidence).

This is why the joint rule **caps numbers at yellow** regardless of the model's reported confidence. Always cross-check numbers against the source video before quoting.

8c. Domain vocabulary failures

The LLM is general-purpose Llama-2; it has weak priors for domain-specific vocabulary (medical terms, legal terms, scientific jargon, brand names, niche tech). The visual encoder may emit a feature sequence that is consistent with the domain term — but without language-context support, the LLM picks a more common English word instead. The LLM judge analysis found ~19 % of segments show clear domain-vocabulary confusion where a topic hint would help. DIY/home content shows the highest N-rate (52 %) for exactly this reason: hands-on visual content with project-specific vocabulary.

Defence: if you know the domain ahead of time, the topic prefix / context-injection feature (see roadmap) can help. For the current release, treat domain-vocabulary segments as *Salvage* by default even when the system marks them *Trust*, and verify named terms.

> *Reference for the human-judged numbers in this section:* `docs/evaluation/llm_judge/` *(blind Opus 4.6 judgments on all 1,497 baseline segments) and the context-aware re-evaluation in* `docs/evaluation/llm_judge/context_eval/`.

---

9. Burned videos

Alongside `report.html`, the pipeline produces a *burned video* for each source: a copy of the original with the hypothesis text overlaid on each segment's frames. Use this for quick spot-checking without flipping between the video and the report.

The overlay text is **coloured per word**, using the same rule as the report's *Hypothesis (Confidence)* column:

On-screen colour	Meaning	Reading rule
Green	**TRUST — confidence + beam agreement both high.**	Read normally.
Yellow	**SALVAGE — partial confidence; treat as ambiguous.**	Glance at the lips before quoting.
Red (or purple in some skins)	**STRIP — model was guessing.**	Ignore the word; rely on context.
Grey italic (no per-word colour)	The whole segment is Strip-tier.	Read the segment as a gist hint, not a transcription.

A segment's tier badge is also burned into the corner of the frame (Trust / Salvage / Strip). If you need a frame-accurate clip for a client demo, the burned video is the asset to use.

Watch with CC (in-browser preview)

On the **Complete** screen there's also a **"Watch with CC"** button. It plays the original video in a small in-page player and overlays the hypothesis text as captions, one segment at a time, coloured by trust band the same way the report and burned videos are. Useful for a quick review before downloading and sharing — no ffmpeg burn needed.

The captions live in a sidecar called `whole_video_cc.json` next to `report.html`, generated automatically when the pipeline completes. v1 maps wall-clock time to segment captions linearly (no per-word timestamps), so a fast talker may see a caption a beat ahead of the mouth movement.

---

10. Troubleshooting

Most of the issues you'll hit on a Windows + Docker Desktop + RTX 5090 laptop are documented in the internal deployment-lessons memory. The five highest-frequency cases are summarised here. If you hit something not covered, email the operator the outputs listed in §10e.

10a. Port 8080 busy

Symptom: PowerShell shows `Bind for 0.0.0.0:8080 failed` or the browser opens to a different application.

Cause: another process (often a previous VSP container, or a local dev server) is already on port 8080.

Fix, in order:

1. In PowerShell: `docker ps` — if you see another `vsp` container, stop it: `docker stop vsp`.
2. `wsl --shutdown` and then restart Docker Desktop. This clears wedged port proxies.
3. If port 8080 is held by a non-Docker process, `Get-NetTCPConnection -LocalPort 8080` shows the PID. Stop that process or restart the laptop.

10b. "no kernel image is available for execution on the device"

Symptom: decode crashes with the literal string `RuntimeError: no kernel image is available for execution on the device`.

Cause: the running container's PyTorch is cu124 (no native sm*120 support for Blackwell). For most operations, cu124's compute*90 PTX JITs successfully to sm_120, but some ops (notably `layer_norm`) refuse and raise this error.

Fix: you are on the wrong image tag. The correct production image has the cu128 PyTorch + `nvidia/` namespace package directories copied into the LLM venv. Check `docker images` — the supported tag is whatever was shipped in the kit's `image.tag` file. If you've applied a custom patch, roll back to the shipped tag.

10c. JIT looks frozen but isn't

Symptom: PowerShell window is silent for 10–15 minutes during the first decode. Browser shows the *Processing* screen at 95 %+ with no movement.

Cause: PyTorch is JIT-compiling PTX kernels to native sm_120 SASS for the RTX 5090. Compilation runs on the CPU and emits no progress output.

Fix: **wait.** This is normal behaviour on the first decode after a fresh image install, after a JIT-cache wipe, or after a PyTorch version upgrade. Subsequent decodes use the cached kernels (cached to `%USERPROFILE%\.nv` outside the container) and skip the pause entirely.

Confirm it isn't actually frozen by checking GPU utilisation in Task Manager → Performance → GPU. During JIT compile the GPU sits near 0 % and one CPU core is at ~100 %.

10d. Browser shows "Connection refused" but `docker ps` shows the container running

Cause: Docker Desktop's port-proxy is wedged (a common Windows issue, especially after Windows updates or sleep/resume).

Fix: in PowerShell:

```
wsl --shutdown
```

Then right-click the Docker Desktop tray icon → Restart. Wait ~30 seconds, refresh the browser.

If that doesn't fix it, also verify you're on `127.0.0.1:8080` and not `localhost:8080`. The two are not interchangeable on Windows; Docker Desktop's IPv6 forwarding is unreliable.

10e. How to send logs to support

When emailing support, attach the following two items:

1. The current container log. Capture it with:

```
docker logs vsp > vsp.log 2>&1
```

2. The PowerShell transcript file (if the installer enabled one) from `%USERPROFILE%\vsp-logs\`.

Five extra one-liners that diagnose ~95 % of *"the server is up but I can't reach it"* situations — paste the output of all five:

```
docker ps -a
docker logs vsp --tail 60
docker port vsp
Test-NetConnection 127.0.0.1 -Port 8080 -InformationLevel Quiet
docker exec vsp python3 -c "import urllib.request; print(urllib.request.urlopen('http://127.0.0.1:8080/',timeout=5).status)"
```

The last one is decisive: if it returns `200` from inside the container but the browser can't reach the same URL, the problem is on the Windows networking side (port-proxy wedged, firewall, antivirus). If it errors inside the container, the server itself didn't start.

---

11. Privacy and air-gap

Nothing leaves the laptop. The pipeline is offline by design:

- The container has no outbound network calls. All models (Llama-2, visual encoder, k-means, Whisper) are baked into the image and loaded from local disk.
- Whisper runs locally (`openai/whisper` weights bundled in the image), not the OpenAI API.
- No telemetry, no analytics, no cloud upload of videos, transcripts, or reports.
- The image is air-gap-deployable: it ships as a single `docker save` tarball, installs from a USB stick, and never requires internet during setup or operation.
- Your videos remain in `%USERPROFILE%\vsp-input`; results remain in the container's `outputs\` (mounted to your host filesystem via Docker Desktop). Both are local-disk paths.

The only file that should ever leave the laptop is whatever **you choose** to email out — the report HTML, the CSV, the burned video, or the logs requested in §10e. The decision is in your hands at every step.

Document version: 2026-05-25. Pipeline version: stage-8 joint-rule + MBR-default (May 2026). Hardware target: Windows 11, RTX 5090 (Blackwell, sm_120), driver ≥570. Image: single-tag, air-gapped Docker container.